

Chapter 2 – Creating and Calling a Function

SLIDE 1 — Creating and Calling a Function

Definition

A function is a reusable block of code that performs a particular task.

Creating a function means writing the instructions. Calling a function means asking the program to execute those instructions.

Analogy

Think of a tea-making machine:

- You create the machine once.
- Whenever you need tea, you call the machine.
- The machine performs the same task again.

Create once → Call whenever needed

Small Example

```
None
void makeTea() {
    printf("Tea is ready");
}
```

Here, `makeTea()` is created, but it will run only when it is called.

SLIDE 2 — Why Are Functions Used?

Why do we use functions?

Functions are used to:

1. Avoid repetition – Write the same code only once.

2. Improve readability – Make the program easier to understand.
3. Divide large programs – Break a big task into smaller tasks.
4. Make debugging easier – Find and fix errors in one section.
5. Reuse code – Call the same function many times.
6. Improve maintenance – Change the function in one place.

Important Statement ★

A good function should perform one clear and specific task.

Base → Professional

Base Level:

None

```
void greet() {  
    printf("Hello");  
}
```

Professional Level:

None

```
void calculateTotal(int price, int quantity) {  
    int total = price * quantity;  
    printf("%d", total);  
}
```

The professional function is more useful because it accepts data and performs a meaningful task.

Coding Guideline 📌

PEP 8 is a style guide for Python, not C. For C programming, follow similar clean-code habits:

- Use meaningful function names.
- Keep functions short.
- Use proper indentation.
- Add comments when needed.
- Give each function one clear responsibility.

SLIDE 3 — Can Functions Be Used in a Greedy Algorithm?

What is a Greedy Algorithm?

A greedy algorithm solves a problem by choosing the best option available at each step.

It focuses on the current best choice instead of checking every possible choice.

Can a Function Be Used?

Yes. A function can be used inside a greedy algorithm.

The function can perform tasks such as:

- Selecting the best item.
- Finding the minimum value.
- Choosing the next activity.
- Checking the available options.

Example Idea

In a coin-selection problem:

```
None
int chooseCoin() {
    return 10;
}
```

The function selects a coin, while the greedy algorithm decides which coin should be selected.

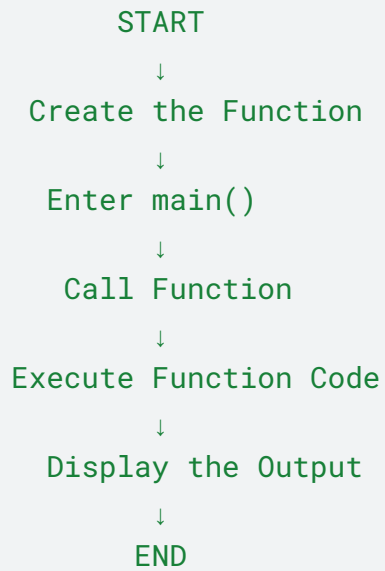
Important Difference

- Function: A reusable block of code.
- Greedy algorithm: A problem-solving strategy.
- A greedy algorithm can be written using one or more functions.

SLIDE 4 — Flowchart: Creating and Calling a Function

Function Execution Flow

None



Code Example

None

```
#include <stdio.h>

void greet() {
    printf("Hello Student");
}

int main() {
    greet();
    return 0;
}
```

Output

None

Hello Student

Explanation

1. The function `greet()` is created.

2. The program enters `main()`.
3. `greet()` is called.
4. The function executes its code.
5. The output is displayed.

Remember ★

Defining a function does not execute it. Calling the function executes it.

SLIDE 5 — Witty Recap

The Function Story 🎬

Imagine a college student preparing for a hackathon:

1. Create the Function The student writes the code once.
2. Call the Function The student uses the code whenever needed.
3. Execute the Function The code performs its task.
4. Get the Output The task is completed successfully.

Funny Memory Trick 😂

Function is like your best friend: You create the friendship once, call them whenever you need help, and they perform the task!

Questions

1. Theory MCQ

What is the purpose of calling a function in C?

A. To delete the function B. To execute the function C. To rename the function D. To stop the program

💡 Hint: Calling means asking the function to run.

📖 Glossary: Execute — To run the instructions written inside a function.

Answer: B. To execute the function

2. Output Guess

None

```
#include <stdio.h>

void greet() {
    printf("Hello");
}

int main() {
    greet();
    return 0;
}
```

What is the output?

💡 Hint: The function is called using `greet()` ;.

📖 Glossary: Call — Using a function name to run its code.

Answer:

None

Hello

3. Fill the Missing Code

None

```
#include <stdio.h>

----- greet() {
    printf("Welcome");
}
```

💡 Hint: Use the keyword when the function returns no value.

📖 Glossary: void — A keyword that indicates no value is returned.

Answer: `void`

4. Jumbled Sentence

Arrange the words correctly:

function / calling / executes / the code / A

💡 Hint: Start with “Calling”.

📖 Glossary: Calling — Asking a function to perform its task.

Answer: Calling a function executes the code.

5. Fill the Missing Code

None

```
#include <stdio.h>

void add() {
    printf("Addition");
}

int main() {
    -----;
    return 0;
}
```

💡 Hint: Write the function name followed by parentheses.

📖 Glossary: Parentheses () — Brackets used when defining or calling a function.

Answer:

None

```
add()
```